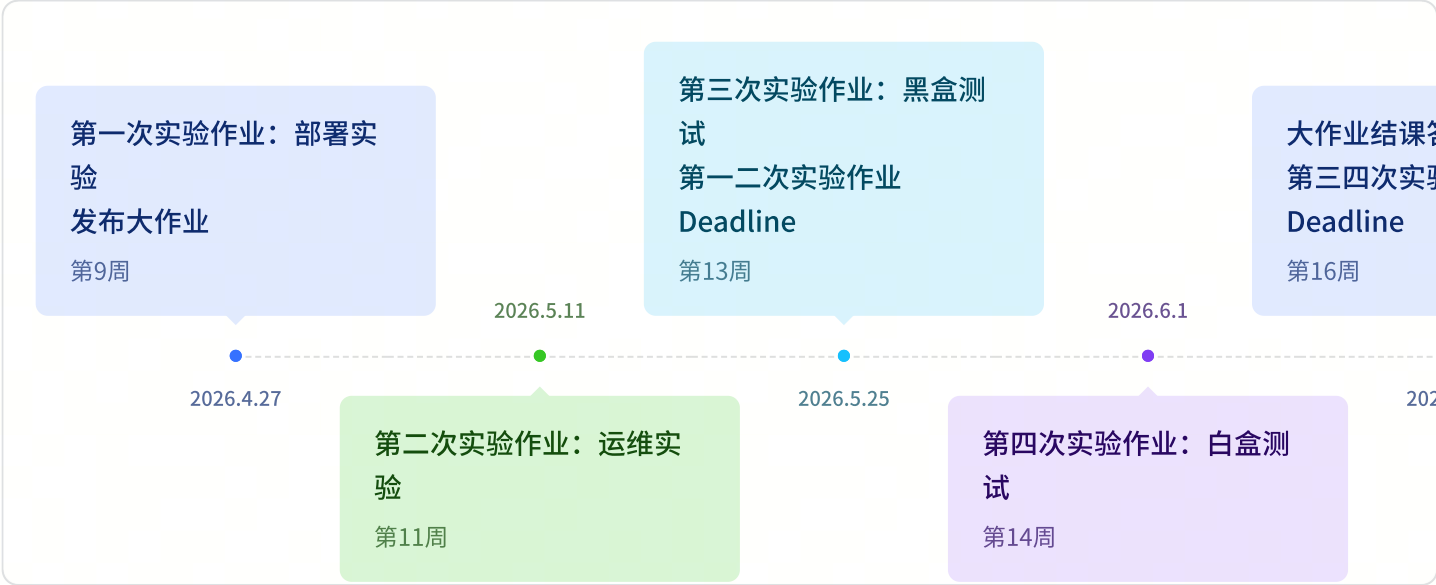


软件测试与维护（2026年春）大作业要求

重要时间节点安排：



大作业内容

本次大作业围绕微服务系统的部署、测试和维护进行实现，大致分为四个阶段：

阶段一：部署微服务系统 & 论文选择与阅读

- 论文地址如下：
- KPI Anomaly Detection: [KPI Anomaly Detection](#)
 - Log Anomaly Detection: [Log Anomaly Detection](#)
 - Trace Anomaly Detection: [Trace Anomaly Detection](#)
 - Failure Diagnosis: [Failure Diagnosis](#)

目的：
在自己的电脑上部署一个开源微服务系统，并搭建minikube集群，学习使用Docker容器化技术。通过选择并阅读两篇关于异常检测与故障诊断的相关论文，为后续的实验做准备，并为论文复现做铺垫。

- 准备工作：**
- Docker 环境准备：**
安装并配置 Docker，确保能够通过 Docker 拉取并运行镜像。熟悉容器的基本操作（如构建、运行、停止等）。

- **Minikube 集群搭建：**

在本地机器上搭建 Minikube 集群，确保可以通过 kubectl 命令管理 Kubernetes 集群。了解如何部署和管理集群中的微服务应用。

- **选择并阅读论文：**

围绕异常检测与故障诊断相关研究方向，选取两篇相关论文进行阅读与分析，理解其主要研究内容、算法设计与实验内容，为后续复现工作奠定基础。

微服务系统部署：

- **选择开源微服务系统：**

部署 SockShop，下载并部署在本地 Minikube 集群中。确保各个微服务组件正常运行。

- **容器化部署：**

使用 Docker 容器化各个微服务，确保每个服务都能够在 Docker 容器内独立运行，并通过 Kubernetes 进行编排和管理。

- **验证部署：**

验证微服务系统是否正常运行。可以通过访问微服务的 API、查看服务日志、监控容器状态等方式确保系统正常运行。

论文选择与阅读：

- **论文选择：**

从异常检测与故障诊断的相关领域中，选择两篇有一定实验基础的论文。确保论文中描述的实验方法能够在微服务系统上进行复现。

- **论文阅读：**

认真阅读论文，重点关注其提出的算法、实验设置、数据处理及结果分析部分。准备好在后续阶段进行复现。

阶段二：使用 Prometheus & Grafana 工具进行数据采集与维护

目的：

在已部署的微服务系统中配置 Prometheus 监控工具，用于采集和监控系统性能数据。通过持续采集监控数据，为后续故障检测与诊断提供可靠的数据支持。随后，利用 ChaosMesh 注入故障，并结合 Prometheus 与 Grafana 进行数据采集、展示和分析，为后续论文复现做好准备。

准备工作：

- **安装与配置 Prometheus：**

在 Minikube 集群中安装并配置 Prometheus 监控工具。确保 Prometheus 能够正确地抓取各微服务的性能指标，并与微服务系统正常对接。

- **配置 Prometheus 数据源：**

配置 Prometheus 监控系统与微服务之间的连接，设置适当的抓取间隔和目标服务。确保 Prometheus 能够实时采集微服务系统的关键性能数据（如请求数量、响应时间、错误率等）。

- **安装 Grafana：**

配置 Grafana 工具，确保其能够与 Prometheus 配合使用，展示微服务系统的监控数据。熟悉 Grafana 仪表盘的设计与使用，能够创建自定义视图以便查看系统状态。

- **部署与验证 ChaosMesh**

了解 ChaosMesh 故障注入工具，并在 Minikube 集群中完成 ChaosMesh 的部署与配置。随后进行故障注入测试，验证 ChaosMesh 是否能够成功注入故障。

- **数据采集：**

利用 Prometheus 采集微服务系统在正常运行状态和故障注入场景下的性能数据，并结合 Grafana 进行可视化展示与分析，为后续论文复现提供数据支撑。

阶段三：使用 Selenium & JMeter 工具进行测试

目的：使用 JMeter 和 Selenium 进行自动化性能和功能测试，模拟真实用户行为并评估微服务系统在不同负载和交互场景下的性能和稳定性。

准备工作：

- 安装并配置 JMeter。
- 安装并配置 Selenium 环境。
- 确认 SockShop 微服务系统已正常部署。

Selenium 功能测试：

- 使用 Selenium 编写自动化脚本，模拟用户在前端界面上的操作，如登录、浏览商品、下单等。
- 配置 Selenium 脚本进行页面加载测试、按钮点击、表单提交等交互，确保前端功能正常。
- 在不同的浏览器中运行测试脚本，验证系统在不同环境下的兼容性。
- 记录页面加载时间、交互响应时间等指标，模拟实际用户操作场景。

JMeter 性能测试：

- 在 JMeter 中创建测试计划，设置线程组模拟多个虚拟用户进行并发请求。
- 配置 HTTP 请求，针对 SockShop 中的不同微服务模块（如订单服务、用户服务等）发送请求。
- 设置不同的并发用户数、持续时间等，模拟不同的负载场景。
- 监控性能指标（如响应时间、吞吐量、错误率等），分析系统在高负载下的表现。

阶段四：用采集的数据运行根据论文复现的算法

目的：使用 ChaosMesh 故障注入工具进行故障注入，利用 Prometheus+Grafana 采集的监控数据，复现论文中的算法，分析数据并验证算法效果。

准备工作：

- 确保 ChaosMesh 故障注入工具已经成功部署，并且可以成功注入故障。
- 确保 Prometheus+Grafana 监控系统已成功配置，并且数据已正常采集。
- 导出相关的性能指标数据（如CPU使用率、内存使用情况、请求延迟等）。

数据预处理+算法实现：

- 根据算法的要求，进行必要的数据预处理。
- 复现算法，使用采集的数据运行算法，分析算法的输出结果

评估与优化：

- 评估算法的效果，若结果不理想，可以尝试调整算法参数或选择其他算法。
- 根据实验结果，撰写优化建议（如算法精度、计算效率等方面的提升）。

要求+评分标准

分组：5-6 人一组

大作业评分标准（3档）：

1. 第一档：完全只在已提供的 **SockShop 部署指南** 的基础上完成部署、监控和维护的大作业。
2. 第二档：自行选择其他更复杂的开源微服务系统([TrainTicket](#)、[Online-Boutique](#)、[Hotel-Reservation](#)、[Social-Network](#)、[Media-Microservices](#))进行部署、监控和维护。
3. 第三档：在第二档的基础上，能够完成该项目中一到两个的**微服务开发**。

加分项：

1. 加分项一：在基础复现两篇异常检测与故障诊断相关论文的基础上，进一步选择**更多论文**进行复现与对比。
2. 加分项二：封装**智能体**自动进行智能运维（可自主选择实现故障注入与验证、异常检测、根因分类、故障恢复等操作中的一项或多项）（具体流程可参考：[👁 使用智能体进行智能运维](#)）。

大作业展示：

1. 微服务系统介绍

介绍所部署的微服务系统，例如 SockShop 或小组自行选择的其他开源微服务项目。如果进行了微服务开发，还需要展示开发部分，说明所开发的微服务功能及其与其他服务的交互。

2. 项目测试展示

使用 Selenium 和 JMeter 对所部署的微服务项目进行测试，展示测试结果，评估系统的性能、响应时间和稳定性。

3. 故障注入与监控

使用 ChaosMesh 进行故障注入，模拟系统中的故障情景，并结合 Prometheus 和 Grafana 进行实时监控与可视化展示，监控系统在故障发生时的表现。

4. 异常数据采集与分析

对小组选择的相关论文中的算法进行复现，使用采集的故障注入后产生的异常数据，展示算法在异常检测与故障诊断中的效果，验证算法在实际环境中的应用表现。

5. 加分项展示（如有）

如果完成了加分项内容，需要展示额外复现论文的对比结果，或展示智能体在运维任务中的实现流程、关键功能与运行效果。

6. 各个成员的贡献

展示小组成员的分工及具体贡献，原则上每位小组成员都需要参与展示。

成员互评：要求各个成员相互打分，加权确定每位同学的大作业得分。

大作业提交内容：

1. 大作业报告

- 提交 PDF 格式报告。
- 项目源码需开源至 GitHub，并在报告中注明仓库链接。

2. 展示汇报 ppt